

LAPORAN UJIAN TENGAH SEMESTER

KOMPUTASI PARALEL DAN TERDISTRIBUSI

ANT WORLD — HYBRID PARALLEL ANT COLONY SIMULATION

Hybrid Computing for Real-World Simulation and Data Processing



Disusun oleh:

Ripki Maulana (247006111139)

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS SILIWANGI

2026

BAGIAN A — KONSEP DAN DESAIN

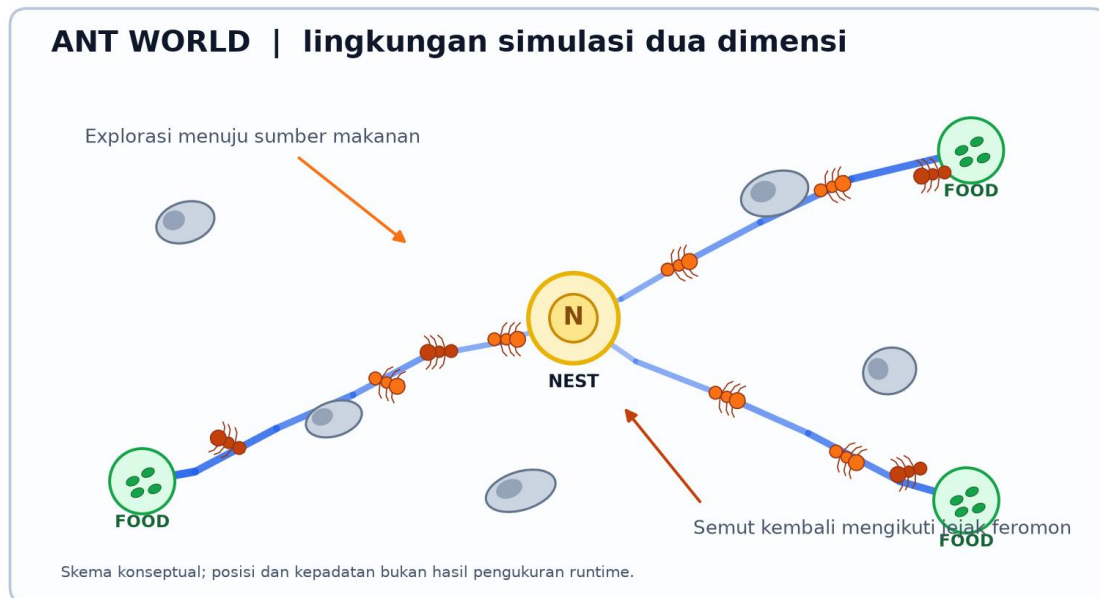
A1. Ide Project dan Permasalahan

ANT WORLD adalah simulasi berbasis agen pada grid dua dimensi. Sejumlah semut bergerak dari sarang untuk menjelajahi lingkungan, menemukan makanan, lalu kembali ke sarang. Lingkungan memuat sarang (nest), sumber makanan (food), rintangan (obstacle), dan lapisan feromon (pheromone). Tiap langkah waktu disebut iterasi; pada setiap iterasi, semut memilih posisi tetangga yang valid berdasarkan feromon dan heuristik jarak, memperbarui keadaan dirinya, serta dapat mengambil makanan atau menyelesaikan perjalanan. Prinsip gerak dan komponen ruang ditunjukkan secara konseptual pada Gambar 1; posisi agen pada ilustrasi bukan data hasil eksperimen.

Permasalahan yang dimodelkan adalah bagaimana sekelompok agen sederhana dapat menjelajah lingkungan dan secara kolektif membentuk jalur pencarian makanan. Perilaku ini berkaitan dengan swarm intelligence: pola kelompok muncul dari interaksi banyak agen yang mengikuti aturan lokal, bukan dari satu pengendali yang merencanakan seluruh lintasan. Feromon berfungsi sebagai jejak tidak langsung yang memengaruhi keputusan berikutnya, sementara penguapan feromon mencegah jejak lama bertahan tanpa batas.

Simulasi dipilih karena memuat banyak keputusan agen yang dapat dihitung secara terpisah pada suatu snapshot keadaan, lalu digabungkan untuk memperbarui keadaan global. Karakter tersebut cocok untuk menunjukkan hybrid computing: process membagi kerja komputasi ant, sementara thread mengelola operasi file dan pembaruan dashboard yang cenderung menunggu I/O. Project ini bertujuan mengamati aliran data, konsistensi hasil serial dan hybrid, dampak overhead paralelisasi, serta metrik waktu dan throughput pada beberapa konfigurasi.

Keterkaitan dengan tema “Hybrid Computing for Real-World Simulation and Data Processing” terletak pada penggunaan simulasi agen untuk memodelkan proses kolektif dan pengolahan data runtime secara bersamaan. Simulasi ini merupakan model edukatif, bukan klaim bahwa perilaku semut yang disederhanakan merepresentasikan ekosistem biologis secara lengkap.



Gambar 1. Ilustrasi konseptual simulasi ANT WORLD pada lingkungan grid dua dimensi; bukan screenshot atau hasil pengukuran runtime.

A2. Konsep Hybrid Computing

Hybrid computing pada project berarti mengombinasikan model memori bersama melalui threads dan model proses terpisah melalui ProcessPool. ThreadPoolExecutor memanfaatkan beberapa thread dalam satu proses untuk menjalankan tugas seperti menulis checkpoint, menulis statistik/CSV, dan meneruskan state ke dashboard. Ketika thread menunggu operasi file atau jaringan lokal, thread lain dapat melanjutkan tugas yang siap. Karena pekerjaan tersebut terutama I/O-bound, thread menghindari biaya membuat proses terpisah untuk setiap operasi.

ProcessPoolExecutor menjalankan fungsi worker komputasi pada proses Python berbeda. Pembagian ini sesuai dengan perhitungan posisi dan keputusan banyak semut yang CPU-bound: pada CPython, thread biasa tidak memberi eksekusi bytecode Python paralel secara umum karena GIL; beberapa proses memiliki interpreter dan GIL masing-masing. Pada project ini worker menerima argumen berupa state semut per chunk dan snapshot dunia, menghitung hasil lokal, lalu mengembalikan struktur hasil ke proses utama.

Satu paradigma saja kurang sesuai untuk dua jenis kerja yang berbeda. ThreadPool saja cocok untuk I/O, tetapi tidak ideal sebagai cara memperoleh paralelisme CPU untuk loop Python. ProcessPool saja dapat menjalankan CPU work, tetapi pekerjaan file/dashboard tidak memerlukan ruang alamat terisolasi dan akan menambah overhead proses serta serialisasi. Kombinasi memungkinkan pemetaan masing-masing jenis tugas ke mekanisme yang lebih sesuai, meskipun biaya koordinasi tetap perlu diukur.

MPI tidak dipakai dalam implementasi utama. Program berjalan pada satu mesin, tidak membagi simulasi ke node melalui jaringan, dan tidak menggunakan communicator atau MPI rank. Karena itu, analisis MPI lintas-node tidak diklaim sebagai hasil project.

A3. Pembagian Beban

Komponen	Jenis beban	Teknologi	Alasan
Langkah semut: pilihan tetangga, bobot probabilitas, state baru	CPU-bound	ProcessPoolExecutor	Chunk semut dihitung pada proses worker terpisah; hasil lokal kemudian dikembalikan.
Checkpoint JSON	I/O-bound	ThreadPoolExecutor	Penulisan file dapat berjalan sebagai tugas asynchronous tanpa membuat process pool khusus I/O.
Statistik JSON dan CSV	I/O-bound	ThreadPoolExecutor	Operasi berorientasi file; dapat di-submit sebagai futures.
Pembaruan dashboard / state HTTP lokal	I/O-bound	ThreadPoolExecutor + thread server	Penerusan state dan request web menunggu operasi I/O lokal.
Sinkronisasi dan penggabungan state global	Koordinasi / serial pada main process	Main process + barrier futures	Menjaga satu keadaan world canonical dan mencegah update global yang tidak konsisten.



Gambar 2. Komponen utama lingkungan simulasi ANT WORLD.

A4. Parameter Personal NIM

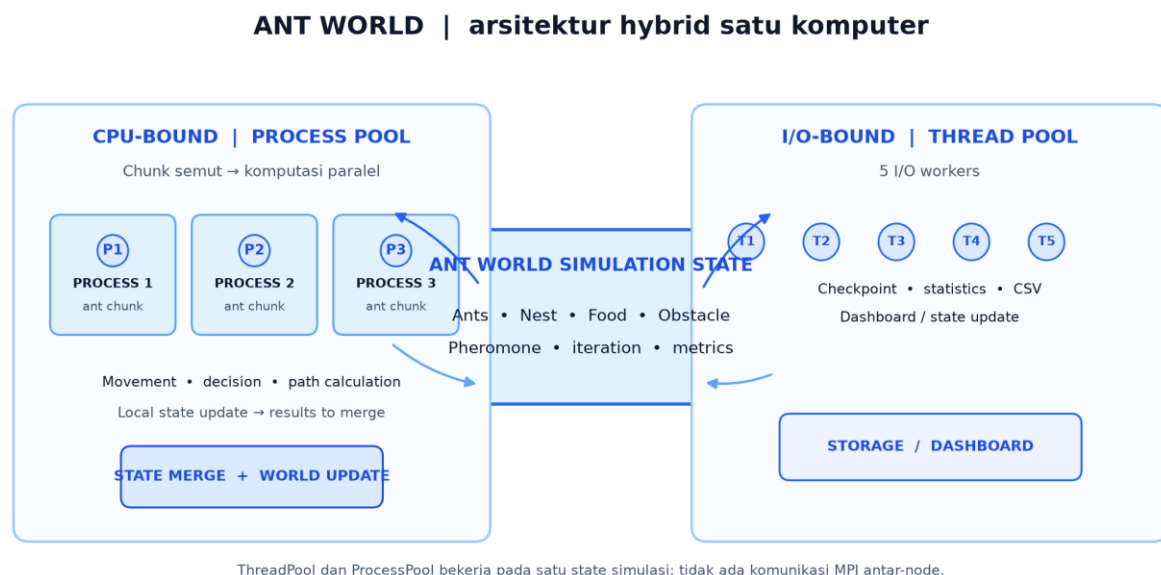
NIM 247006111139 menjadi sumber parameter personal yang dikonfigurasi pada project. Parameter tersebut tidak diubah dalam laporan ini.

Parameter	Aturan / sumber	Nilai project
NIM	Identitas mahasiswa	247006111139
Threads	$39 \bmod 4 + 2$	5 thread I/O
Processes	$61 \bmod 3 + 2$	3 worker proses
Data	139×10 semut	1.390 ants
Random seed	Seed menggunakan nilai NIM	247006111139
Ukuran world	modul konfigurasi	100×100 cell
Sumber makanan	modul konfigurasi	10 food sources
Rintangangan	modul konfigurasi	20 obstacles
Maksimum iterasi konfigurasi	modul konfigurasi	1.000 iterasi (kecuali argumen runtime mengubahnya)

Keterangan formula dua digit: berkas modul konfigurasi mengambil nilai dua digit terakhir 39 untuk thread, dan dua digit di posisi tengah 61 untuk jumlah proses, sesuai komentar/konfigurasi project. Nilai efektif yang digunakan tetap 5 dan 3 sebagaimana ditentukan.

A5. Arsitektur Sistem

Arsitektur memisahkan jalur komputasi dan jalur I/O. Pada jalur komputasi, main process menyiapkan input worker, menunggu semua future, lalu menjadi satu-satunya pihak yang menerapkan hasil ke world global. Pada jalur I/O, pengelola tugas I/O membuat ThreadPoolExecutor untuk operasi file dan pembaruan state dashboard. Dashboard server itu sendiri menggunakan ThreadingHTTPServer dan thread server latar untuk melayani HTTP lokal; fungsi tersebut dibedakan dari worker CPU ProcessPool.



Gambar 3. Diagram arsitektur hybrid ANT WORLD yang menunjukkan state simulasi, ProcessPool CPU-bound, dan ThreadPool I/O-bound.

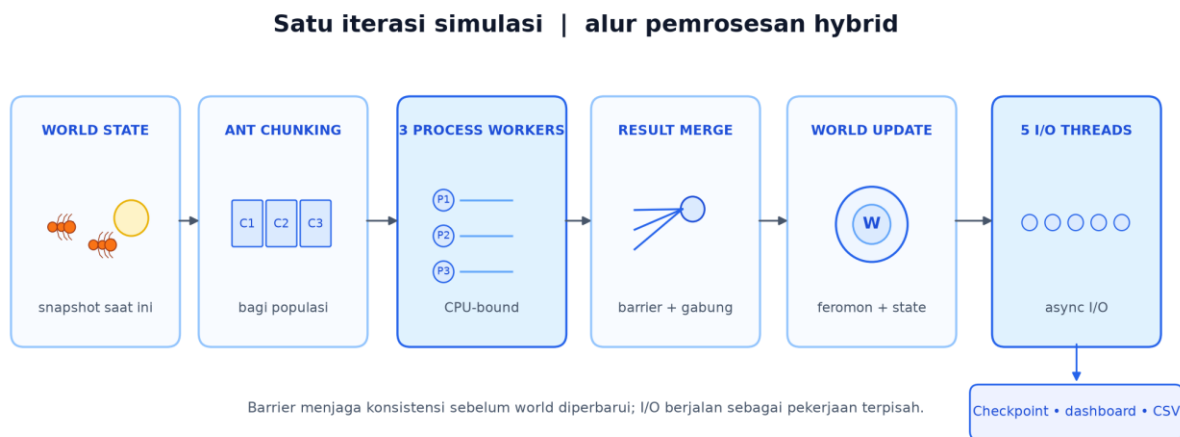
A6. Alur Simulasi

- 1) Konfigurasi memuat NIM, seed, jumlah semut, jumlah thread/proses, ukuran world, sumber makanan, rintangan, dan iterasi.
- 2) Seed diteruskan ke generator angka acak dan world dibangun; setiap semut diberi identitas serta keadaan awal di sarang.
- 3) Simulasi menyimpan state canonical: world, semut, iterasi, status, jumlah makanan terkumpul, dan ringkasan path.
- 4) Pada setiap iterasi, main process membagi daftar semut menjadi chunk berdasarkan jumlah worker dan membentuk snapshot world.
- 5) Setiap chunk bersama snapshot dikirim ke worker ProcessPool melalui executor.submit; worker menghitung keputusan gerak semut berdasarkan snapshot.

- 6) Worker mengembalikan state semut terbaru dan event lokal seperti upaya mengambil makanan, feromon yang perlu ditaruh, perjalanan selesai, serta `chunk_id`.
- 7) Main process menunggu future (barrier), mengurutkan hasil dengan `chunk_id`, kemudian menggabungkan state/event secara deterministik.
- 8) State semut dan world global diperbarui; pengambilan makanan, deposit di sarang, dan penguapan feromon diterapkan setelah hasil worker tersedia.
- 9) ThreadPool menangani checkpoint, statistik/CSV, dan pembaruan dashboard sesuai operasi yang diminta dan opsi runtime.
- 10) Proses berulang hingga batas iterasi; program menghitung metrik akhir, menyimpan ringkasan, lalu menutup worker pool, I/O pool, dan server.
- 11) Launcher dashboard menunggu tombol START, mengukur baseline serial, lalu menjalankan mode hybrid. Benchmark batch menjalankan serial dan hybrid berpasangan untuk setiap konfigurasi, tanpa server dashboard aktif. Metode ini menjaga batas pengukuran benchmark terpisah dari overhead layanan dashboard.

A7. Alur Satu Iterasi

Flowchart ini memperlihatkan satu iterasi hybrid: main process membagi semut ke dalam chunk dan mengirimkannya bersama snapshot world ke ProcessPool. Setelah seluruh worker selesai, hasil digabung secara deterministik; state semut dan world diperbarui, sementara ThreadPool menangani tugas I/O seperti checkpoint dan pembaruan dashboard.



Gambar 4. Alur pemrosesan hybrid pada satu iterasi simulasi ANT WORLD.

A8. Desain Dashboard

Dashboard web lokal menampilkan peta simulasi pada canvas dengan legenda untuk semut, feromon, makanan, sarang, dan rintangan. Opsi layer mengatur visibilitas semut, jejak feromon, serta rintangan. Panel ringkasan memuat status dan iterasi, jumlah semut, makanan tersedia/terkumpul, panjang lintasan, dan durasi iterasi.

Dashboard ANT WORLD menyediakan panel hybrid computing yang memperlihatkan pembagian tugas I/O-bound dan CPU-bound, jumlah thread/process, data, dan seed.

Layanan dashboard menyediakan tampilan lokal serta API untuk membaca state dan menerima permintaan mulai. State bersama dilindungi lock; server HTTP berjalan pada thread latar, sementara pembaruan state dikirim melalui ThreadPool I/O.

BAGIAN B — IMPLEMENTASI

B1. Konfigurasi Sistem

Konfigurasi personal yang digunakan pada pengujian terdiri dari lima thread I/O, tiga worker proses, 1.390 semut, dan seed NIM. Perangkat dan versi interpreter dirinci pada Bagian C. Tidak ada MPI karena seluruh eksekusi dilakukan pada satu komputer.

B2. Implementasi Simulasi

Parameter	Nilai aktual
NIM / seed	247006111139
Thread I/O	5
Worker CPU	3 process
Ants	1.390
World	100 × 100
Food sources / obstacles	10 / 20
Nest position	(50, 50)
Maximum iterations	1.000 pada config; dapat dioverride CLI
Pheromone decay / deposit	0,1 / 10,0
Alpha / beta	1,0 / 2,0

World direpresentasikan oleh grid NumPy terpisah untuk makanan (bilangan bulat), obstacle (boolean), dan feromon (float). Nest berada pada koordinat konfigurasi. Inisialisasi memilih lokasi makanan dan obstacle yang tidak bertumpuk dengan sarang atau satu sama lain. Posisi valid membatasi koordinat ke grid dan menolak obstacle; tetangga yang dapat dipilih adalah hingga delapan sel di sekitar semut.

Setiap Ant menyimpan id, posisi, status membawa makanan, perilaku, path, dan generator acak. Jika membawa makanan, semut menggunakan jarak ke nest sebagai heuristik; jika mencari, ia mengeksplorasi tetangga. Bobot kandidat dipengaruhi feromon ditambah konstanta kecil dan heuristik jarak berpangkat beta. Probabilitas dinormalisasi sebelum pemilihan posisi. Saat mencapai sumber makanan semut mengambil satu unit; saat kembali ke nest, makanan dihitung sebagai terkumpul dan jalur dapat menghasilkan deposit feromon.

Simulasi melakukan langkah semut, menyelesaikan pengambilan/deposit, menggabungkan jejak yang selesai, menguapkan feromon dan menaikkan penghitung iterasi. Metrik state mencakup iterasi, status, makanan terkumpul/sisa, jumlah perjalanan, jumlah semut membawa makanan, pheromone average/maximum, serta ringkasan panjang jalur.

```
# Inti serial per iterasi (diringkas dari komponen simulasi)
for ant in self.ants:
    ant.move(self.world, ALPHA, BETA, self.iteration)
    ant.try_collect_food(self.world)
    if ant.try_deposit_food():
        self.food_collected += 1
# Gabungkan jejak setelah keputusan semua semut
for ant in completed_ants:
    ant.deposit_pheromone_trail(self.world, PHEROMONE_DEPOSIT)
self.world.decay_pheromones(PHEROMONE_DECAY)
self.iteration += 1
```

B3. ProcessPool

Pool proses dibuat untuk digunakan kembali sepanjang iterasi simulasi. Pada setiap langkah, semut dibagi menjadi sejumlah chunk sesuai jumlah worker. State semut dikemas sebagai data yang dapat dikirim ke proses lain, bersama snapshot ukuran world, grid makanan, rintangan, feromon, dan posisi sarang. Setiap worker menjalankan perhitungan lokal pada chunk yang diterimanya. Alur state dan jalur CPU/I/O divisualisasikan pada Gambar 4.

Setiap worker menghitung state lokal tanpa menulis langsung ke objek world utama. Main process mengumpulkan future, menangani exception, mengurutkan output berdasarkan chunk_id, lalu menggabungkan state ant dan event. Mekanisme barrier ini menghindari update global sebagian saat iterasi belum selesai, tetapi membatasi kemajuan iterasi berikutnya sampai worker yang paling lambat selesai. Data snapshot/state yang berulang dikirimkan ke worker, sehingga serialization, IPC dan biaya copy merupakan overhead potensial.

```
# Bentuk inti coordinator (komponen pemrosesan paralel)
work_items = [(ant_chunks[i], world_snapshot, iteration, i, seed)
               for i in range(len(ant_chunks))]
futures = [executor.submit(worker CPU, item) for item in
work_items]
results = [future.result() for future in futures] # barrier
results.sort(key=lambda r: r['chunk_id'])
# hasil kemudian digabung dan diterapkan pada state utama
```

B4. ThreadPool

Pengelola I/O membentuk ThreadPoolExecutor dengan lima worker. Operasi asynchronous mengirim penulisan checkpoint, statistik, snapshot dashboard, atau CSV sebagai task dan mengembalikan Future. Ketika hasil harus dipastikan telah tersedia, proses utama menunggu Future; tugas lainnya dapat berjalan sambil pemanggil melanjutkan pekerjaan.

```
# Pola submit I/O dari komponen I/O
def save_checkpoint_async(self, state, filepath):
    return self.executor.submit(self._save_checkpoint, state,
                                filepath)

def write_stats_async(self, stats, filepath):
    return self.executor.submit(self._write_stats, stats, filepath)

def write_csv_async(self, data, filepath, headers=None):
    return self.executor.submit(self._write_csv, data, filepath,
                                headers)
```

Selain lima thread I/O tersebut, layanan HTTP menggunakan thread server tersendiri dan server multithread untuk menangani koneksi. Keduanya bukan bagian dari angka lima worker pada ThreadPoolExecutor.

B5. Sinkronisasi dan Penggabungan State

Setelah seluruh worker mengembalikan Future, proses utama menunggu hasil lengkap, meneruskan exception jika ada, mengurutkan hasil berdasarkan identitas chunk, lalu menerapkan state ant dan event pada world canonical. Pengambilan makanan, pengembalian makanan ke sarang, deposit feromon, dan penguapan dilakukan pada urutan yang menjaga hasil serial/hybrid tetap deterministik. Barrier membuat setiap iterasi menunggu worker paling lambat; pola ini sederhana untuk menjaga konsistensi, tetapi berpotensi membatasi skalabilitas.

B6. Serial Baseline

Baseline serial menjadi pembanding speedup. Untuk tiap konfigurasi, seed, jumlah semut, jumlah iterasi, dan logika simulasi dijaga sama antara serial dan hybrid. Benchmark mencatat waktu simulasi serial dan hybrid berpasangan. Waktu awalisasi/persiapan pool tidak dipisahkan pada tabel, sehingga waktu terukur merepresentasikan batas timer implementasi benchmark.

Rumus yang digunakan project: $S = T_{\text{serial}} / T_{\text{parallel}}$. Jika $S > 1$, parallel lebih cepat pada workload yang dibandingkan; $S = 1$ setara; $S < 1$ berarti lebih lambat daripada serial. Perbandingan bermakna hanya ketika batas pengukuran (setup, baseline, dashboard, checkpoint, warmup) disamakan atau dijelaskan.

B7. Pengukuran Performa

Empat pasangan pengukuran mencatat execution time, speedup, efficiency, throughput, jumlah makanan, serta kecocokan hasil deterministik. Grafik terkontrol memperbandingkan thread, process, dan konfigurasi berdasarkan workload yang sesuai.

Metrik	Definisi implementasi	Catatan interpretasi
Execution time	Wall time pengukuran simulasi melalui MetricsCollector.	Nyatakan entry point dan apakah setup/serialization/dashboard termasuk.
Speedup	$S = T_{\text{serial}} / T_{\text{parallel}}$.	T_{serial} dan T_{parallel} wajib workload setara.
Efficiency	$E = S / \text{jumlah_process} \times 100\%$.	Menggunakan jumlah proses sebagai n; ini bukan efisiensi berdasarkan thread I/O.
Throughput	ants / execution_time, satuan ants/s.	Dalam kode, numerator jumlah semut, bukan jumlah semut $\times$ iterasi; gunakan definisi kode ini.

Grafik membandingkan pasangan dengan faktor lain tetap: jumlah thread pada 3 proses dan 1.390 ants; jumlah process pada 5 thread dan 1.390 ants; serta speedup/efficiency untuk empat konfigurasi. Semua grafik pada bagian ini menggunakan hasil benchmark 100 iterasi yang sama dengan Tabel 5.

B8. Dashboard dan Output

Output launcher menampilkan identitas, mode hybrid, jumlah thread/process, jumlah ants, seed, iterasi, dan metrik runtime. Screenshot terminal pada Gambar 5 merupakan sesi demo tersendiri dengan 5 threads, 3 processes, 1.390 ants, 100 iterations, baseline serial sekitar 5,10 s, hybrid sekitar 3,7875 s, dan speedup sekitar 1,3478 $\times$. Nilai tersebut bukan pengganti benchmark komparatif pada Tabel 5; seluruh analisis konfigurasi pada Bagian C menggunakan Tabel 5.

```
PowerShell — ANT WORLD hybrid run

PS D:\Kuliah\Season 5\KPT\UTS> .\venv\Scripts\python.exe .\main.py --mode hybrid --iterations 100

-----
ANT WORLD - HYBRID COMPUTING (HYBRID PARALLEL)
-----
Student Name : Ripki Maulana
NIM          : 247006111139
Threads (I/O) : 5
Processes (CPU): 3
Ants         : 1390
Seed         : 247006111139
Iterations   : 100
-----

Measuring serial baseline (same ants, iterations, threads)...
Serial baseline: 5.10s
-----

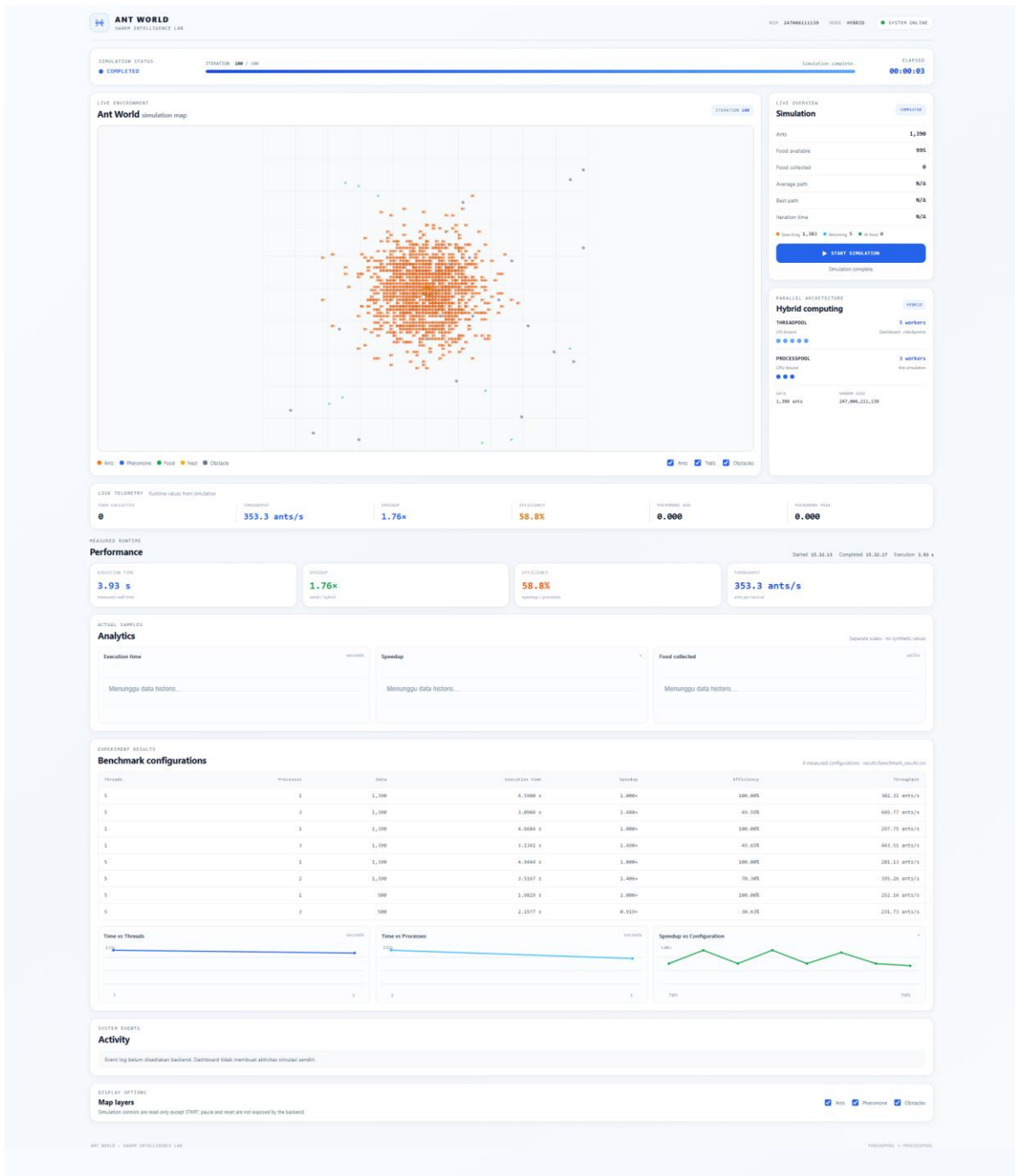
Iteration 0: Food= 0, Trips= 0, AvgPheromone= 0.00
Iteration 50: Food= 0, Trips= 0, AvgPheromone= 0.00
-----

PERFORMANCE RESULT
-----
Execution Time : 3.7875 seconds
Throughput     : 367.00 ants/second
Food Collected : 0
Total Trips    : 0
Best Path Length : N/A
Avg Path Length : N/A
Speedup        : 1.3478x (T_serial / T_parallel)
Efficiency     : 44.93% (speedup / 3 * 100%)
Baseline measured : 5.1049s (serial, same workload)
-----

Stats saved to .\results\report_screenshots\terminal_capture_stats.json
```

Gambar 5. Output terminal saat ANT WORLD berjalan dalam mode hybrid; hasil runtime terpisah dari pengukuran benchmark pada Tabel 5.

Dashboard lokal digunakan untuk memantau peta dan state simulasi, status/iterasi, jumlah ants dan makanan, serta parameter ThreadPool/ProcessPool. Panel hybrid menunjukkan tugas I/O-bound dan CPU-bound; telemetry menampilkan execution time, throughput, speedup, dan efficiency pada sesi dashboard. Screenshot ini merupakan bukti visual runtime, sedangkan Tabel 5 tetap menjadi dasar perbandingan eksperimen. Pada sesi screenshot, simulasi berstatus completed dan food collected tercatat 0. Ilustrasi komponen lingkungan terdapat pada Gambar 2; screenshot dashboard aktual tetap ditampilkan terpisah pada Gambar 6.



Gambar 6. Dashboard ANT WORLD setelah simulasi selesai, memperlihatkan peta, panel hybrid computing, dan telemetry runtime.

Dashboard lokal menampilkan peta simulasi, status/iterasi, jumlah ant dan makanan, konfigurasi ThreadPool/ProcessPool, seed, serta telemetry performa. Pada screenshot, status simulasi telah selesai dan food collected tercatat 0. Dashboard berfungsi sebagai visualisasi runtime; nilainya tidak menggantikan data perbandingan pada tabel benchmark.

Panel hybrid computing pada Gambar 6 memperlihatkan ThreadPool untuk tugas I/O-bound dan ProcessPool untuk simulasi CPU-bound, beserta jumlah worker, ukuran data, dan seed. Area telemetry/performance pada gambar yang sama menampilkan metrik sesi dashboard; angka benchmark yang digunakan untuk analisis konfigurasi tetap mengacu pada Tabel 5.

BAGIAN C — HASIL EKSPERIMEN DAN ANALISIS

C1. Lingkungan Pengujian

Komponen	Spesifikasi yang terverifikasi
CPU	Intel(R) Core(TM) i3-4160 CPU @ 3.60 GHz
Physical core	2
Logical processor	4
RAM	7.92 GB terdeteksi oleh sistem
OS	Windows 10 Pro 64-bit, build 19045
Python aktif saat pemeriksaan	Python 3.14.3
Project dependencies	numpy, matplotlib, pandas (); concurrent.futures/multiprocessing dari standard library
Personal parameters	5 I/O threads, 3 process workers, 1.390 ants, seed 247006111139

Interpreter yang digunakan saat pengukuran adalah Python 3.14.3. Pustaka eksternal yang tercantum sebagai dependensi project adalah NumPy, Matplotlib, dan pandas; executor menggunakan concurrent.futures serta multiprocessing dari standard library.

C2. Metodologi Pengujian

Pengukuran membandingkan baseline serial dan mode hybrid dengan seed sama. Dalam setiap pasangan, jumlah ant dan iterasi sama; benchmark batch dijalankan tanpa server dashboard agar batas timer antarvariasi konsisten.

Benchmark dijalankan selama 100 iterasi per konfigurasi. Variasi thread membandingkan 1 dan 5 pada 3 proses serta 1.390 ants; variasi process membandingkan 2 dan 3 pada 5 thread serta 1.390 ants; variasi data membandingkan 500 dan 1.390 ants pada 5 thread dan 3 proses. Setiap hybrid dipasangkan dengan baseline serial pada workload/seed/iterasi yang sama.

Run	Threads	Processes	Data (ants)	Iterations	Keterangan
Konfigurasi personal	5	3	1.390	100	Acuan personal
Variasi thread	1	3	1.390	100	Thread diubah; lainnya tetap
Variasi process	5	2	1.390	100	Process diubah; lainnya tetap
Variasi data	5	3	500	100	Data diubah; lainnya tetap

C3. Konfigurasi Eksperimen

Empat konfigurasi dijalankan selama 100 iterasi. Konfigurasi personal: 5 thread, 3 process, 1.390 ants. Variasi thread: 1 thread, 3 process, 1.390 ants. Variasi process: 5 thread, 2 process, 1.390 ants. Variasi data: 5 thread, 3 process, 500 ants. Setiap pasangan dibandingkan terhadap baseline serial dengan seed, iterasi, dan ukuran data identik.

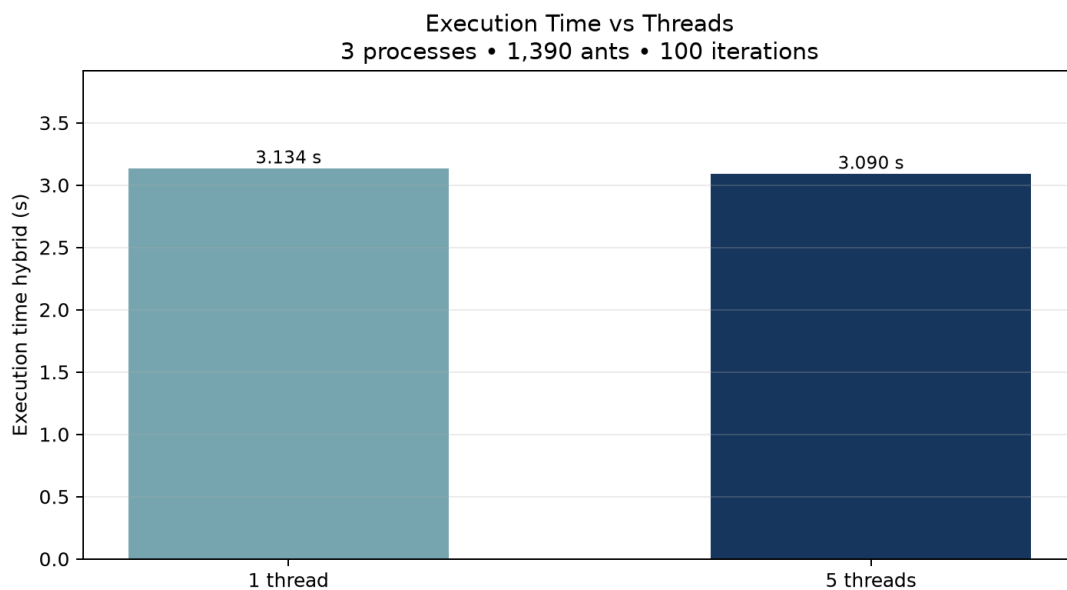
C4. Hasil Benchmark

Tabel 5. Hasil benchmark aktual 100 iterasi; baseline serial dan hybrid dipasangkan per workload.

No.	Thread	Process	Data (ants)	T serial (s)	T hybrid (s)	Speedup	Efficiency	Throughput (ants/s)
1	5	3	1.390	4,597952	3,090437	1,4878×	49,59%	449,77
2	1	3	1.390	4,668423	3,134073	1,4896×	49,65%	443,51
3	5	2	1.390	4,944367	3,516699	1,4060×	70,30%	395,26
4	5	3	500	1,982886	2,157669	0,9190×	30,63%	231,73

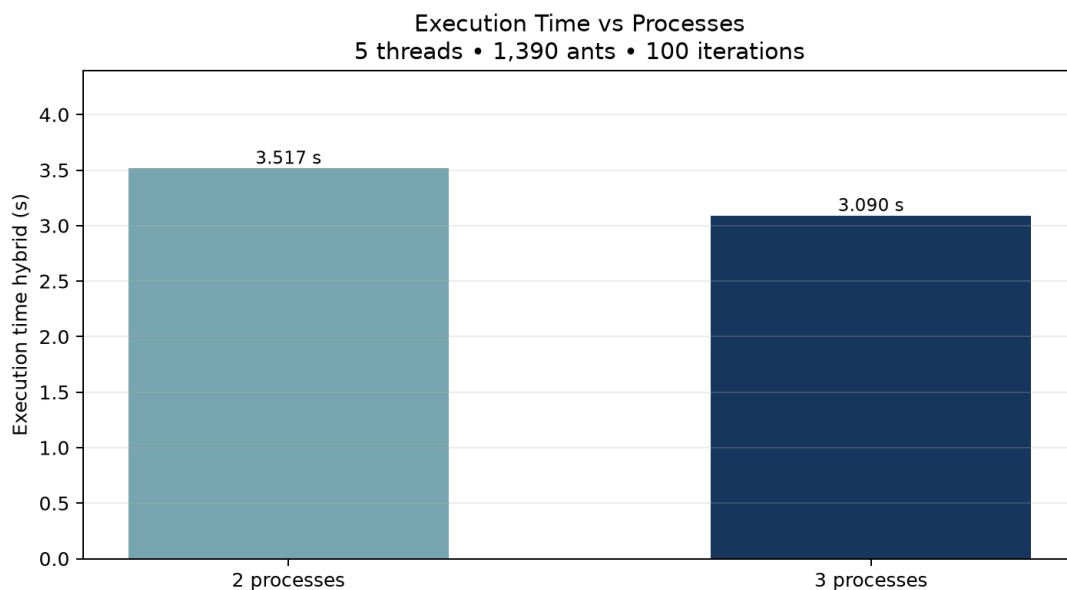
Setiap konfigurasi menjalankan 100 iterasi dengan seed yang sama antara serial dan hybrid; program menyatakan hasil deterministik cocok. Food collected tercatat 0 pada seluruh konfigurasi. Untuk 1.390 ants, konfigurasi personal mencatat waktu hybrid 3,0904 s, speedup 1,4878×, efficiency 49,59%, throughput 449,77 ants/s. Perubahan thread ke 1 menghasilkan waktu 3,1341 s; dua proses memberi 3,5167 s. Data 500 ants mempunyai waktu hybrid 2,1577 s dan speedup 0,9190×. Nilai food collected sebesar 0 menunjukkan bahwa pada konfigurasi dan durasi pengujian belum terdapat perjalanan lengkap dari sumber makanan kembali ke nest. Kondisi ini tidak mengubah perbandingan waktu eksekusi karena seluruh konfigurasi menggunakan workload dan jumlah iterasi yang sama, tetapi menjadi keterbatasan pada aspek perilaku simulasi.

C5. Grafik Eksperimen



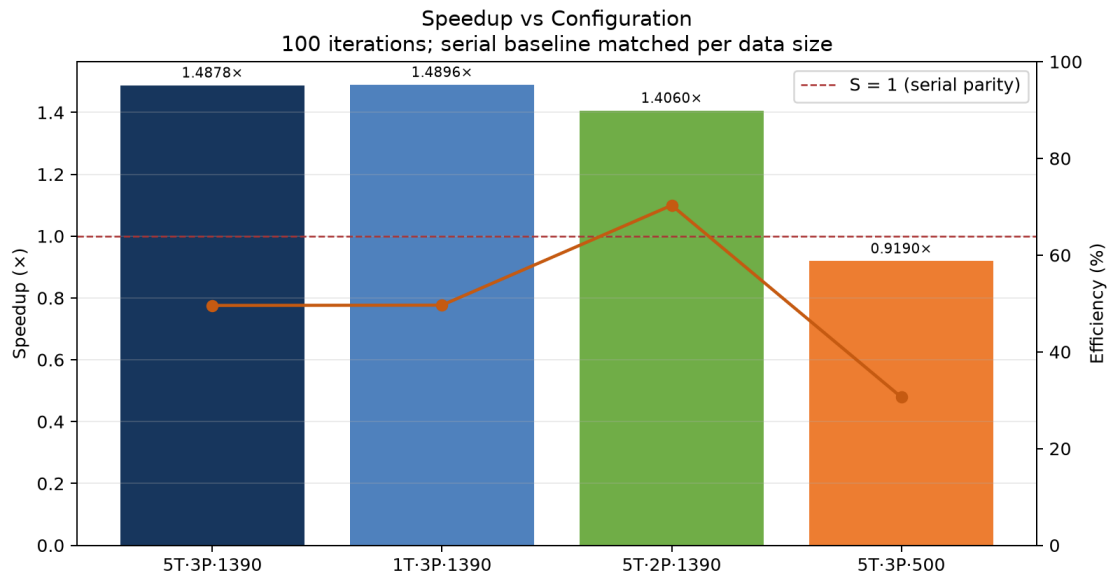
Gambar 7. Execution time terhadap jumlah thread.

Dengan 3 process dan 1.390 ants, waktu hybrid turun dari 3,134073 s pada 1 thread menjadi 3,090437 s pada 5 thread. Penurunannya relatif kecil; pada workload ini jumlah thread I/O bukan faktor dominan terhadap waktu komputasi ant.



Gambar 8. Execution time terhadap jumlah process.

Dengan 5 thread dan 1.390 ants, waktu hybrid turun dari 3,516699 s pada 2 process menjadi 3,090437 s pada 3 process. Pada pasangan terukur ini, penambahan worker CPU memperbaiki waktu, tetapi kesimpulan berlaku untuk workload dan mesin pengujian ini.



Gambar 9. Speedup dan efficiency berdasarkan konfigurasi.

Konfigurasi personal 5 thread, 3 process, dan 1.390 ants menghasilkan speedup 1,4878×. Pada 500 ants, speedup 0,9190× menunjukkan hybrid lebih lambat daripada serial untuk workload tersebut. Speedup tidak bertambah linear seiring sumber daya.

Grafik waktu terhadap thread dan process membandingkan pasangan dengan faktor lain tetap. Grafik speedup menampilkan keempat konfigurasi; efficiency ditampilkan pada sumbu sekunder. Nilai pada grafik diturunkan langsung dari tabel hasil benchmark 100 iterasi.

C6. Analisis Speedup

Pada konfigurasi personal (5 thread, 3 process, 1.390 ants), waktu hybrid tercatat 3,0904 s dan speedup 1,4878× terhadap baseline 4,5980 s. Variasi thread 1 menghasilkan speedup 1,4896×; variasi process 2 menghasilkan 1,4060×. Dengan 500 ants, speedup turun menjadi 0,9190×: hybrid lebih lambat daripada serial untuk workload yang lebih kecil. Hasil ini mendukung bahwa manfaat proses bergantung pada ukuran kerja.

Pada workload 500 ants, waktu hybrid 2,1577 s lebih panjang daripada baseline serial 1,9829 s sehingga speedup 0,9190×. Untuk 1.390 ants, speedup berada di atas satu. Perbedaan ini konsisten dengan overhead koordinasi yang lebih terasa pada workload kecil. Pengujian tidak memisahkan biaya startup, pickling, IPC, transfer snapshot, barrier, dan merge; sehingga penyebab rinci belum dapat dipastikan tanpa profiling.

C7. Analisis Efficiency

Efficiency dihitung sebagai $E = S / \text{jumlah proses} \times 100\%$. Untuk konfigurasi personal nilainya 49,59%; variasi thread 49,65%; variasi dua process 70,30%; workload 500 ants 30,63%. Efficiency adalah rasio speedup terhadap resource proses, bukan persentase utilisasi CPU. Mesin memiliki 2 physical core dan 4 logical processor, sementara konfigurasi personal memakai 3 worker CPU.

C8. Analisis Throughput

Throughput hybrid yang terukur: 449,77 ants/s untuk konfigurasi personal; 443,51 ants/s untuk 1 thread; 395,26 ants/s untuk 2 process; dan 231,73 ants/s pada 500 ants. Kode mendefinisikan throughput sebagai jumlah ant dibagi waktu eksekusi. Karena jumlah data berbeda pada run keempat, interpretasikan bersama ukuran workload.

C9. Analisis Bottleneck

Faktor potensial	Hubungan dengan kode	Status bukti
Process startup / scheduling	Hybrid membuat dan mematikan executor; benchmark menggunakan ProcessPool per konfigurasi.	Mungkin menambah overhead; kontribusi belum diukur terpisah.
Pickling / IPC / state transfer	Ant chunk dan snapshot grid berupa argumen/result yang diserahkan lintas proses.	Pola ada di source; besar biaya perlu profiling.
Barrier / synchronization	Main process menunggu semua future sebelum merge dan update state.	Mekanisme eksplisit pada source; keterlambatan worker paling lambat menahan iterasi.
Merge serial	Hasil ant dan event digabung serta diterapkan di main process.	Pola tampak di kode; proporsi waktu belum tersedia.
CPU saturation	3 CPU worker pada mesin 2 physical core / 4 logical processor.	Dapat menjadi faktor, tetapi dampaknya perlu hasil aktual.
Memory	Snapshot grid dan state ant disalin/diubah menjadi struktur serializable.	Potensi biaya transfer; penggunaan memori benchmark belum tercatat.
I/O	Checkpoint/statistics/dashboard ditangani task I/O; fitur bergantung mode/opsi run.	Waktu I/O terpisah tidak tercatat di CSV benchmark.

C10. Kesesuaian ThreadPool dan ProcessPool

Pemetaan konseptual sudah tepat: thread dipakai untuk aktivitas yang menunggu file/dashboard, sedangkan process digunakan untuk komputasi langkah semut CPU-bound yang ditulis dalam Python. ThreadPool mengurangi keharusan membuat proses untuk kerja I/O; ProcessPool menghindari ketergantungan pada satu GIL untuk paralelisme CPU antarproses. Namun ketepatan pilihan arsitektur tidak otomatis berarti waktu total lebih baik: untuk kerja yang terlalu kecil, biaya process dan transfer state dapat lebih besar daripada komputasi yang dibagi.

C11. Analisis Dashboard

Seperti ditunjukkan pada Gambar 6, dashboard menampilkan peta dan legenda semut, feromon, makanan, sarang, serta rintangan. Panel status, iterasi, jumlah ants, makanan, konfigurasi hybrid, seed, telemetry, dan performa digunakan untuk memantau runtime. Dashboard adalah visualisasi; angka pembandingan eksperimen tetap mengacu pada Tabel 5.

BAGIAN D — ANALISIS AKHIR, KESIMPULAN DAN SARAN

D1. Analisis Performa

Pada workload personal 5 thread, 3 process, dan 1.390 ants, waktu hybrid 3,0904 s adalah yang terkecil di antara tiga konfigurasi dengan 1.390 ants. Perubahan thread menjadi 1 menghasilkan 3,1341 s, sedangkan dua process menghasilkan 3,5167 s. Workload 500 ants mencatat speedup di bawah satu. Karena setiap konfigurasi hanya dijalankan sekali, ini perbandingan observasional, bukan bukti signifikansi statistik.

Hybrid berpotensi menguntungkan jika total kerja CPU cukup besar untuk menutup biaya pembagian dan transfer data, sementara tugas I/O dapat diproses asinkron. Serial dapat lebih baik ketika pekerjaan kecil, chunk terlalu halus, atau hasil harus sering disinkronkan. Penentuan konfigurasi paling baik harus menunggu benchmark terkontrol, bukan berdasarkan jumlah worker semata.

D2. Analisis Bottleneck

Barrier, serialisasi/pickling state semut dan snapshot, IPC, serta merge di main process tetap merupakan biaya koordinasi yang terlihat dari desain. Pengujian 500 ants dengan speedup 0,9190× menunjukkan seluruh biaya hybrid lebih besar daripada manfaat komputasi paralel pada workload itu. Pada personal data 1.390 ants, speedup di atas satu menunjukkan manfaat paralel lebih besar secara total dalam run tersebut; komponen biaya tertentu belum diukur terpisah.

D3. Hubungan Teori dan Hasil

ThreadPool sesuai untuk I/O-bound karena operasi baca/tulis dan request dapat menunggu tanpa memerlukan satu proses interpreter baru per tugas. ProcessPool sesuai untuk beban CPU-bound Python karena proses mempunyai ruang eksekusi/interpreter terpisah sehingga GIL tidak membatasi seluruh pekerja pada satu interpreter. Sebaliknya, process membutuhkan penjadwalan OS dan transfer data, sedangkan thread berbagi address space namun perlu perhatian pada state bersama.

Amdahl's Law menyatakan bahwa fraksi serial membatasi speedup maksimum: $S(N)=1/[s+(1-s)/N]$. Pada project, merge global, update pheromone/world, barrier, dan bagian koordinasi adalah kandidat komponen yang tidak meningkat sebanding dengan worker. Tanpa mengukur fraksi serial, laporan tidak menetapkan nilai numerik Amdahl. Skalabilitas membutuhkan porsi kerja paralel besar dan biaya komunikasi yang tidak tumbuh lebih cepat daripada manfaatnya. Hasil speedup historis kurang dari satu mengingatkan bahwa penambahan worker sendiri tidak menjamin akselerasi.

D4. Kesimpulan

ANT WORLD menerapkan hybrid computing pada satu komputer: ProcessPoolExecutor membagi perhitungan gerak semut yang CPU-bound; ThreadPoolExecutor menangani checkpoint, statistik/CSV, dan pembaruan dashboard yang I/O-bound. Proses utama mempertahankan state global dan menunggu barrier sebelum merge dan update world. MPI tidak digunakan karena project tidak berjalan pada cluster multi-node.

Empat benchmark 100 iterasi selesai dan determinism serial-hybrid terverifikasi. Food collected tercatat 0 pada seluruh konfigurasi; karena itu simulasi belum menunjukkan perjalanan semut yang berhasil mencapai sarang kembali dalam durasi uji ini.

Dengan demikian, arsitektur hybrid berjalan dan menghasilkan speedup lebih dari satu pada workload personal 1.390 ants dalam pengujian 100 iterasi ini; pada 500 ants hasilnya di bawah satu. Kesimpulan performa ini dibatasi satu kali run per konfigurasi dan perlu diulang untuk mengukur variasi waktu. Pelajaran utama adalah paradigma paralel harus dipilih berdasar jenis beban serta biaya koordinasi, bukan banyaknya worker semata.

D5. Saran Pengembangan

- 1) Ulangi beberapa kali setiap konfigurasi dengan seed, iterasi, dan workload tetap untuk memperoleh median/rentang waktu yang lebih andal.
- 2) Dokumentasi eksekusi terminal dan dashboard telah disertakan pada Bagian B sebagai bukti visual implementasi.
- 3) Kurangi serialization dengan mengevaluasi ukuran payload dan strategi snapshot; jangan mengoptimalkan tanpa profiling.
- 4) Pertimbangkan chunk size/scheduling yang lebih baik bila hasil menunjukkan load imbalance; saat ini chunking bersifat statis dan barrier menunggu seluruh worker.
- 5) Pertimbangkan shared memory untuk data grid besar hanya jika profiling membuktikan transfer snapshot sebagai bottleneck dan desain konsistensi dapat dijaga.
- 6) MPI baru relevan bila simulasi benar-benar dibagi ke beberapa node dan tersedia kebutuhan/lingkungan cluster; GPU hanya layak diuji jika operasi vektor dapat dipetakan dan overhead transfer data dapat dibenarkan.

DAFTAR PUSTAKA (FORMAT IEEE)

- [1] Python Software Foundation, “concurrent.futures — Launching parallel tasks,” Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/concurrent.futures.html>. Diakses 8 Okt. 2026.
- [2] Python Software Foundation, “multiprocessing — Process-based parallelism,” Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/multiprocessing.html>. Diakses 8 Okt. 2026.
- [3] Python Software Foundation, “threading — Thread-based parallelism,” Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/threading.html>. Diakses 8 Okt. 2026.
- [4] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in Proc. AFIPS Spring Joint Computer Conf., 1967, pp. 483–485.
- [5] M. Dorigo, V. Maniezzo, and A. Coloni, “Ant system: Optimization by a colony of cooperating agents,” IEEE Trans. Syst., Man, Cybern. B, vol. 26, no. 1, pp. 29–41, 1996.
- [6] P. S. Pacheco, An Introduction to Parallel Programming. Burlington, MA: Morgan Kaufmann, 2011.
- [7] Ripki Maulana, “ANT WORLD — Hybrid Parallel Ant Colony Simulation,” implementasi dan catatan pengujian project, 2026.
- [8] Program Studi Informatika, Fakultas Teknik, Universitas Siliwangi, “Naskah Soal Ujian Tengah Semester: Komputasi Paralel dan Terdistribusi,” Semester Ganjil Tahun Ajaran 2026/2027, 24 Sept. 2026.